How Do Graph Signals Affect Recommendation: Unveiling the Mystery of Low and High-Frequency Graph Signals

Feng Liu*
School of Computer Science and Technology
Soochow University
Suzhou, China
fliuliufeng@stu.suda.edu.cn

Hao Cang*
School of Computer Science and Technology
Soochow University
Suzhou, China
hcangada@stu.suda.edu.cn

Huanhuan Yuan
School of Computer Science and Technology
Soochow University
Suzhou, China
hhyuan@stu.suda.edu.cn

Jiaqing Fan
School of Computer Science and Technology
Soochow University
Suzhou, China
jqfan@suda.edu.cn

Yongjing Hao
School of Electronic and Information Engineering
Suzhou University of Science and Technology
Suzhou, China
yjhao@mail.usts.edu.cn

Fuzhen Zhuang
Institute of Artificial Intelligence, Beihang University
State Key Laboratory of Complex & Critical Software Environment
Beihang University
Beijing, China
zhuangfuzhen@buaa.edu.cn

Guanfeng Liu
School of Computing
Macquarie University
Sydney, Australia
guanfeng.liu@mq.edu.au

Pengpeng Zhao†
School of Computer Science and Technology
Soochow University
Suzhou, China
ppzhao@suda.edu.cn

Abstract

Spectral graph neural networks (GNNs) are highly effective in modeling graph signals, with their success in recommendation often attributed to low-pass filtering. However, recent studies highlight the importance of high-frequency signals. The role of low-frequency and high-frequency graph signals in recommendation remains unclear. This paper aims to bridge this gap by investigating the influence of graph signals on recommendation performance. We theoretically prove that the effects of low-frequency and high-frequency graph signals are equivalent in recommendation tasks, as both contribute by smoothing the similarities between user-item pairs. To leverage this insight, we propose a frequency signal scaler, a plug-and-play module that adjusts the graph signal filter function to fine-tune the smoothness between user-item pairs, making it compatible with any GNN model. Additionally, we identify and prove that graph embedding-based methods cannot fully capture the characteristics of graph signals. To address this limitation, a space flip method is introduced to restore the expressive power of graph embeddings. Remarkably, we demonstrate that either low-frequency or high-frequency graph signals alone are sufficient for effective recommendations. Extensive experiments on four public datasets validate the effectiveness of our proposed methods. Code is avaliable at https://github.com/mojosey/SimGCF.

CCS Concepts

• **Information systems → Collaborative filtering**.

Keywords

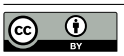
Spectral Graph Neural Networks, Collaborative filtering

ACM Reference Format:
Feng Liu, Hao Cang, Huanhuan Yuan, Jiaqing Fan, Yongjing Hao, Fuzhen Zhuang, Guanfeng Liu, and Pengpeng Zhao. 2026. How Do Graph Signals Affect Recommendation: Unveiling the Mystery of Low and High-Frequency Graph Signals. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD'26), August 9–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780194

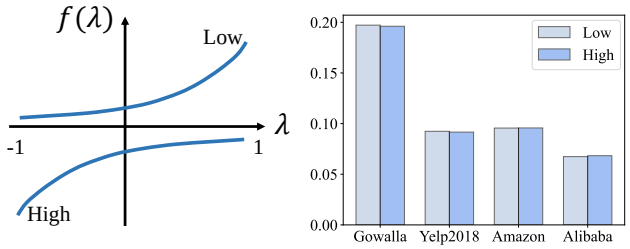
1 Introduction

Graph neural networks (GNNs) have been widely used in collaborative filtering-based recommender systems [7, 9, 10] due to the ability to effectively capture high-level information on graphs. Early GNN-based recommendation primarily rely on spatial domain GNNs [10, 13, 24]. They explore the construction of graph structures, the design of message propagation mechanisms and aggregation strategies from a spatial perspective, achieving remarkable results.

*Feng Liu and Hao Cang contributed equally to this work.
†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD'26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780194



(a) (b)

Figure 1: Analyze the impact of low-frequency and high-frequency graph signals on recommendation. (a) The waveforms of low-pass function in quadrant I and high-pass function in quadrant III. (b) The performance of low-pass and high-pass GNNs on four datasets, evaluated on Recall@20.

Recently, recommendations based on spectral domain GNNs have garnered significant attention [15, 17], focusing on exploring the role of graph signals in recommendation. Researchs mainly include matrix decomposition-based methods [14, 20, 26] and graph embedding-based methods [6, 9, 23]. The former is computationally efficient but difficult to scale, while the latter approximates graph filters through polynomials and is more suitable for large-scale recommendations. Studies have shown that most current GNN-based recommendation models function as low-pass filters [9, 10]. These studies indicate that low-frequency graph signals play a key role in GNN-based recommendation models [18, 20], as they can smooth the similarities between the embeddings of connected nodes. However, a study suggests that high-frequency graph signals also contribute to recommendation [6]. They find that low-frequency and high-frequency graph signals in the training data are strongly linearly correlated with those in the test data, and they propose a graph embedding method based on Jacobi polynomial bases to leverage graph signals of different frequencies.

However, **how do low-frequency and high-frequency graph signals affect recommendation** remains an open question. Limited by the exploration of graph signals across different frequencies in recommender systems, the underlying mechanisms of graph signals in recommendation remain largely unexplored.

In this paper, we explore how graph signals affect recommendation. First, we explore the impact of low and high-frequency signals in recommendation. As shown in Figure 1(a), we utilize the low-frequency graph signal **Low** and the high-frequency graph signal **High** with the same absolute value of waveform for recommendation. As shown in Figure 1(b), their performances are very close, indicating that their impact on recommendations is equivalent. We theoretically prove that low and high-frequency graph signals with the same absolute value of waveform have an equal impact on recommendation. Graph signals affect recommendation by changing the similarity between user-item pairs, the key lies in the waveform of the graph signal filter function. Based on the above experimental and theoretical findings, we introduce a frequency signal scaler constructed with a learnable monomial basis to further refine the waveform of the graph signal filter function in the original GNN model. As a plug-and-play plugin, it can be applied to any GNN to enhance the performance of downstream tasks.

In addiation, we theoretically demonstrate that existing spectral GNNs cannot fully express the characteristics of graph signals. This limitation arises from the fact that current spectral GNNs represent graph signals through graph embeddings [11, 12, 25, 28]. We find that graph signals exhibit four distinct types of features and the graph embedding signals can only capture half of these features, with the other features being hidden. Therefore, we proposes a space flip method to restore the expressiveness of graph embedding signals by flipping the original graph embeddings.

The contributions of this paper are as follows:

- We study the impact of graph signals on recommendation and prove that low-frequency and high-frequency graph signals are equivalent in recommendation, both influencing recommendation performance by altering the similarity between user-item pairs.
- We analyze the differences between graph embedding signals and graph signals, finding that current graph embedding signals cannot fully capture the characteristics of graph signals, and propose a space flip method to enhance their expressive power.
- Extensive experiments on four widely used benchmark datasets demonstrate that our approach effectively improves upon state-of-the-art models.

2 Related Works

2.1 Spectral Graph Collaborative Filtering based on Matrix Decomposition

Spectral graph collaborative filtering based on matrix decomposition primarily investigates the properties of graph signals on user-item graphs derived through singular value decomposition (SVD) or eigenvalue decomposition. GC_CF [20] demonstrates the critical role of low-frequency graph signals in CF and proposes an algorithm that combines linear and ideal low-pass filters to directly filter the interaction matrix. PGSP [14] argues that high-frequency graph signals also play an important role in representing user preferences and introduces a personalized graph signal processing method to incorporate high-frequency signals. SGFCF [26] studies the expressive power of graph signal filters used in current decomposition-based spectral collaborative filtering methods and points out that linear filters cannot fit arbitrary embeddings. Although matrix decomposition-based spectral methods enable simple and efficient filtering of graph signals, they require eigenvalue decomposition, which limits their efficiency in large-scale recommendation. Moreover, they fail to encode structural information into vectors, which restricts their scalability to downstream tasks.

2.2 Spectral Graph Collaborative Filtering based on Graph Embedding

Spectral graph collaborative filtering based on graph embedding approximates graph signal filters by using different polynomial bases. In theory, any graph signal filter can be approximated by a high-order polynomial basis. Early graph embedding-based collaborative filtering models are low-pass graph signal filters. For

example, NGCF [23] and LightGCN [9] are both low-pass filtering graph models based on monomial bases. JGCF [6] points out that high-frequency graph signals are also crucial in recommendation, and proposes a hybrid filtering GCF model based on Jacobi polynomial bases. Current graph embedding methods [1, 3, 8] generally assume that the graph signals represented by embeddings are equivalent to the graph signals, without thoroughly exploring the differences between them. However, graph embedding can only capture part of the graph signal characteristics. Moreover, the impact of graph signals at different frequencies within embeddings on recommendation performance remains an open question.

3 Preliminary

3.1 Spectral Graph Neural Network

We denote the user set as $\mathcal{U}$, the item set as $\mathcal{I}$, and the nodes set as $N = |\mathcal{U}| + |\mathcal{I}|$. The interaction matrix of users and items is $\mathbf{R} \in \mathbb{R}^{|\mathcal{U}|\times|\mathcal{I}|}$ where $\mathbf{R}_{ij} = 1$ if the i-th user and the j-th item have an interaction. The adjacency matrix of the graph in the graph-based recommendation can be expressed as follows:

$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{R} \\ \mathbf{R}^T & \mathbf{0} \end{bmatrix} \tag{1}$$

Spectral GNN performs graph convolution in the Laplacian spectral domain. The Laplacian matrix can be defined as $\mathbf{L} = \mathbf{D} - \mathbf{A}$, where $\mathbf{D} \in \mathbb{R}^{N\times N}$ represents the degree matrix of the graph. Its normalized form can be expressed as $\hat{\mathbf{L}} = \mathbf{I} - \hat{\mathbf{A}}$, where $\hat{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}$. As $\hat{\mathbf{L}}$ is a symmetric positive definite matrix, we can obtain its eigenvalues and corresponding eigenvectors through eigenvalue decomposition, $\hat{\mathbf{L}} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$, where $\mathbf{U} \in \mathbb{R}^{N\times N}$ is the eigenvector and $\mathbf{\Lambda} \in \mathbb{R}^{N\times N}$ is the corresponding eigenvalue, the eigenvalue range of the Laplacian matrix is [0,2]. The graph Fourier transform of the signal $\mathbf{x} \in \mathbb{R}^N$ can be defined as $\hat{\mathbf{x}} = \mathbf{U}^T\mathbf{x}$ and the inverse transform is $\mathbf{x} = \mathbf{U}\hat{\mathbf{x}}$. This transformation enables the formulation of operations such as filtering in the spectral domain. The filtering operation on signal $\mathbf{x}$ can be defined as:

$$\mathbf{y} = f(\hat{\mathbf{L}})\mathbf{x} = \mathbf{U}f(\mathbf{\Lambda})\mathbf{U}^T\mathbf{x} \tag{2}$$

Directly computing $f(\mathbf{\Lambda})$ is difficult due to the eigenvalue decomposition of large-scale matrices is very time-consuming. Therefore, spectral GNN often uses some polynomials to approximate $f(\mathbf{\Lambda})$:

$$f(\mathbf{\Lambda}) = \sum_{i=0}^{n} \theta_i \mathbf{P}_i(\mathbf{\Lambda}) \tag{3}$$

where θ_i is usually a learnable or fixed scalar, $\mathbf{P}_i(\mathbf{\Lambda})$ usually uses various forms of polynomial bases, such as Monomials, Chebyshev polynomials and Jacobi polynomial bases.

3.2 Graph Signal Filter in Recommendation

Different from spectral graph neural networks, GNN-based recommendation typically perform graph signal filtering on the normalized adjacency matrix $\hat{\mathbf{A}}$, influenced by the removal of self-loops as proposed in LightGCN [9]. Consequently, the eigenvalues of $\hat{\mathbf{A}}$ fall within the range of [-1, 1].

Definition 1 *Graph Signal* (GS). The graph signal refers to a series of eigenvalues $\mathbf{\Lambda}$ ranging from [-1,1] obtained by eigenvalue decomposition of the adjacency matrix $\hat{\mathbf{A}}$. It exhibits high-frequency characteristics when the eigenvalue is close to -1 and low-frequency characteristics when it is close to 1. Usually, the spectral operation on the graph signals can be performed by the graph signals filtering function $f(\lambda)$:

$$\mathbf{S}_1 = f(\hat{\mathbf{A}}) = \mathbf{U}f(\mathbf{\Lambda})\mathbf{U}^T = \sum_{i=0}^{n} \alpha_i \mathbf{U}\mathbf{\Lambda}^i\mathbf{U}^T = \sum_{i=0}^{n} \alpha_i \hat{\mathbf{A}}^i \tag{4}$$

Definition 2 *Graph Embedding Signal* (GES). The graph embedding signal refers to compressing the features of the graph signal into vectors by embedding methods, $\mathbf{E} = f(\hat{\mathbf{A}})\mathbf{E}^0$. The features of graph signals are approximately restored by calculating the similarity between vectors:

$$\mathbf{S}_2 = \mathbf{E}\mathbf{E}^T = (f(\hat{\mathbf{A}})\mathbf{E}^0)(f(\hat{\mathbf{A}})\mathbf{E}^0)^T \tag{5}$$

Definition 3 *Graph signal filtering function.* The graph signal filter function $f(\lambda)$ refers to the transformation of the original graph signal $\lambda \in \Lambda$ to obtain a new graph signal, usually approximated by a polynomial bases. Given a set of polynomial coefficients $\alpha_i > 0$, where $i \in [0, n]$, we can obtain four different quadrants of graph signal filtering function, as shown in Figure 2:

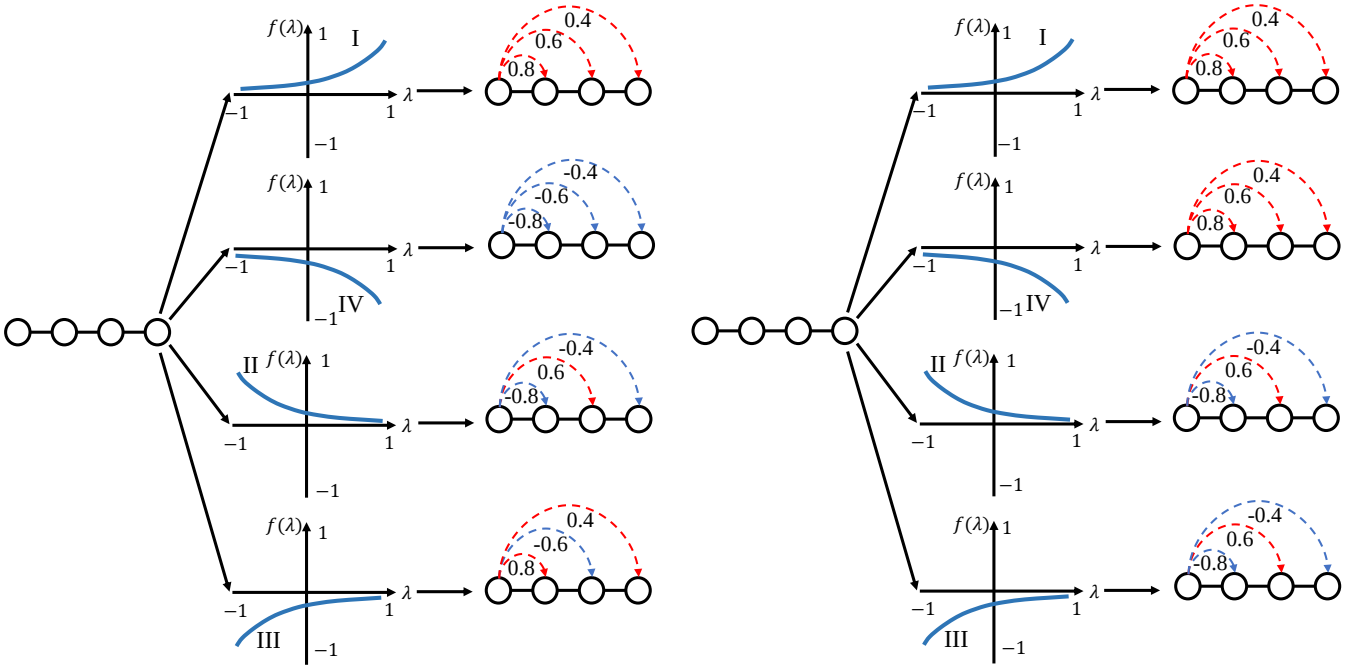
- **Low(I)**, Low frequency filter function in quadrant I: $f(\lambda) = \alpha_0\lambda^0 + \alpha_1\lambda^1 + ... + \alpha_n\lambda^n = \sum_{i=0}^{n} \alpha_i\lambda^i$.
- **Low(IV)**, Low frequency filter function in quadrant IV: $f(\lambda) = -\alpha_0\lambda^0 - \alpha_1\lambda^1 - ... - \alpha_n\lambda^n = \sum_{i=0}^{n} -\alpha_i\lambda^i$.
- **High(II)**, High frequency filter function in quadrant II: $f(\lambda) = \alpha_0\lambda^0 - \alpha_1\lambda^1 + ... + (-1)^n\alpha_n\lambda^n = \sum_{i=0}^{n} (-1)^i\alpha_i\lambda^i$.
- **High(III)**, High frequency filter function in quadrant III: $f(\lambda) = -\alpha_0\lambda^0 + \alpha_1\lambda^1 + ... + (-1)^{n+1}\alpha_n\lambda^n = \sum_{i=0}^{n} (-1)^{i+1}\alpha_i\lambda^i$.

4 How do Graph Signals Work in Recommender Systems?

This section aims to investigate how graph signals function affect recommender systems. Specifically, we first explore the ability of graph embedding signals methods to express graph signals. Then we analyze the key factors that affect recommendations with graph signals of different frequencies. These analyses collectively provide a theoretical basis for understanding the role of graph signal frequency in recommendation performance.

4.1 Is Graph Signal Equal to Graph Embedding Signal?

Collaborative Filtering (CF) assumes that users are similar to the items they have interacted with, and is usually viewed as a link prediction task on a homogeneous graph. Study has shown that low-frequency Graph Embedding Signals (GES) play a key role in this task [20]. Recently, a study shows that there is a strong linear correlation between the high-frequency Graph Signals (GS) of the interaction graphs in the training set and the test set. They proposed a GES method based on the Jacobi polynomial bases to utilize the high-frequency GS [6]. This seems to contradict the assumption of CF. As high-frequency GES cause connected users and items to be dissimilar, while CF assumes that there is similarity between interacting users and items. As shown in Table 1, the experimental results indicate that high-frequency GES have a much smaller impact on the performance compared to low-frequency GES. In fact, we find that there is a huge difference between the



(a) Graph Signals (b) Graph Embedding Signals

Figure 2: The Characteristics of graph signals and graph embedding signals.

GES and the GS, and proved that the GES can only express part of the characteristics of the GS. The high-frequency GS that are beneficial for recommender systems cannot be expressed by GES.

Table 1: The impact of high-frequency graph embedding signals and low-frequency graph embedding signals.

Datasets	High		Low	
	Recall@10	NDCG@10	Recall@10	NDCG@10
Gowalla	0.0217	0.0134	0.1362	0.0987
Yelp2018	0.0097	0.0071	0.0562	0.0453
Amazon-Books	0.0107	0.0087	0.0620	0.0508
Alibaba-iFasion	0.0129	0.0063	0.0450	0.0244

First, we describe the specific characteristics of graph signals and graph embedding signals.

Theorem 1 *Characteristics of low-frequency GS.* The low-frequency GS S^1_{low} exhibits a stable changing characteristic, and the absolute value of the similarity between nodes decays with the increase of the path length. When $f(\lambda) > 0$, all connected nodes will become similar, and when $f(\lambda) < 0$, all connected nodes will become dissimilar. As shown in Figure 2(a), the signal filtering functions $f(\lambda)$ in quadrants I and IV.

Theorem 2 *Characteristics of high-frequency GS.* The high-frequency GS S^1_{high} exhibits the characteristics of frequent changes, and the absolute value of the similarity between nodes decays with the increase of the path length. When $f(\lambda) > 0$, the odd-order nodes will be dissimilar and the even-order nodes will be similar. When $f(\lambda) < 0$, the odd-order nodes will be similar and the even-order nodes will be dissimilar. As shown in Figure 2(a), the signal filtering functions $f(\lambda)$ in quadrants II and III.

Theorem 3 *Characteristics of low-frequency GES.* The low-frequency GES S^2_{low} exhibits a stable changing characteristic, and the absolute value of the similarity between nodes decays with the increase of the path length. When $f(\lambda) > 0$ or $f(\lambda) < 0$, it makes all connected nodes similar. As shown in Figure 2(b), the signal filtering functions $f(\lambda)$ in quadrants I and IV.

Theorem 4 *Characteristics of high-frequency GES.* The high-frequency GES S^2_{high} exhibits the characteristics of frequent changes, and the absolute value of the similarity between nodes decays with the increase of the path length. When $f(\lambda) > 0$ or $f(\lambda) < 0$, it will make the odd-order nodes dissimilar and the even-order nodes similar. As shown in Figure 2(b), the signal filter function $f(\lambda)$ in quadrants II and III.

From the above theorems, we can draw the following theoretical corollary.

Corollary 1 *The GES can only preserve the characteristics of the GS with $f(\lambda) > 0$, while the GES will hide the characteristics of the GS with $f(\lambda) < 0$.*

Proof of Corollary 1 Given a graph signal filter function with $f(\lambda) > 0$ and $f_a(\hat{\mathbf{A}}) = \sum_{i=0}^{n} \alpha_i \hat{\mathbf{A}}^i$, the corresponding graph embedding signal is $\mathbf{S}_a^2 = \mathbf{E}_a\mathbf{E}_a^T = (f_a(\hat{\mathbf{A}})\mathbf{E}^0)(f_a(\hat{\mathbf{A}})\mathbf{E}^0)^T$. Similarly, the corresponding graph signal filtering function for $f(\lambda) < 0$ is $f_b(\hat{\mathbf{A}}) = -f_a(\hat{\mathbf{A}}) = \sum_{i=0}^{n} -\alpha_i \hat{\mathbf{A}}^i$, and the graph embedding signal is $\mathbf{S}_b^2 = \mathbf{E}_b\mathbf{E}_b^T = (f_b(\hat{\mathbf{A}})\mathbf{E}^0)(f_b(\hat{\mathbf{A}})\mathbf{E}^0)^T$. Based on the above conditions, we can derive the following derivation:

$$
\begin{aligned}
\mathbf{S}_a^2 &= \mathbf{E}_a\mathbf{E}_a^T = (f_a(\hat{\mathbf{A}})\mathbf{E}^0)(f_a(\hat{\mathbf{A}})\mathbf{E}^0)^T \\
&= (\alpha_0\hat{\mathbf{A}}^0\mathbf{E}^0 + ... + \alpha_n\hat{\mathbf{A}}^n\mathbf{E}^0)(\alpha_0\hat{\mathbf{A}}^0\mathbf{E}^0 + ... + \alpha_n\hat{\mathbf{A}}^n\mathbf{E}^0)^T \\
&= (-\alpha_0\hat{\mathbf{A}}^0\mathbf{E}^0 - ... - \alpha_n\hat{\mathbf{A}}^n\mathbf{E}^0)(-\alpha_0\hat{\mathbf{A}}^0\mathbf{E}^0 - ... - \alpha_n\hat{\mathbf{A}}^n\mathbf{E}^0)^T \\
&= (-f_a(\hat{\mathbf{A}})\mathbf{E}^0)(-f_a(\hat{\mathbf{A}})\mathbf{E}^0)^T \\
&= (f_b(\hat{\mathbf{A}})\mathbf{E}^0)(f_b(\hat{\mathbf{A}})\mathbf{E}^0)^T = \mathbf{E}_b\mathbf{E}_b^T = \mathbf{S}_b^2
\end{aligned} \tag{6}
$$

The above derivation shows that GES fails to capture the negative sign when representing graph signals with $f(\lambda) < 0$. We assume $\mathbf{X} = f_a(\hat{\mathbf{A}})\mathbf{E}^0$, $-\mathbf{X} = -f_a(\hat{\mathbf{A}})\mathbf{E}^0 = f_b(\hat{\mathbf{A}})\mathbf{E}^0$. In this case, $\mathbf{S}_a^2 = \mathbf{X}^2$ and $\mathbf{S}_b^2 = (-\mathbf{X})^2$. However, the GES corresponding to $f(\lambda) < 0$ should actually be $-\mathbf{X}^2$. Therefore, a negative sign needs to be applied to $\mathbf{S}_b^2$ to recover the correct GES. See the Appendix A for details.

4.1.1 Space Flip. Through the proof of **Corollary 1**, we can easily observe that GES fails to express negative sign when calculating the similarity matrix between nodes, which suppresses some of the properties of GS. Therefore, we need to add a negative sign in front when calculating these features to restore the original properties:

$$
\mathbf{S}_2 = -\mathbf{E}\mathbf{E}^T = (-\mathbf{E})\mathbf{E}^T = \mathbf{E}_f\mathbf{E}^T \tag{7}
$$

This operation can be viewed as a space flip of the original graph embedding $\mathbf{E}$ to obtain a new embedding $\mathbf{E}_f = -\mathbf{E}$, and then calculating its similarity with the original graph embedding $\mathbf{E}$. By space flip, we can recover the hidden high-frequency features of GS in quadrant III. As shown in Table 2, we can find that compared with the features of high-frequency GS in quadrant II, the features of high-frequency GS in quadrant III have a significant impact on the recommendation performance, which is at the same level as the impact of low-frequency GS in quadrant I.

Table 2: The performance of GS under four different quadrants on Recall@10 after space flip.

Datasets	low(IV)	low(I)	high(II)	high(III)
Gowalla	0.0219	0.1362	0.0217	0.1362
Yelp2018	0.0113	0.0562	0.0097	0.0561
Amazon-Books	0.0102	0.0620	0.0107	0.0619
Alibaba-iFasion	0.0130	0.0450	0.0129	0.0453

4.2 How do Low-frequency and High-frequency Graph Signals Affect Recommendation?

From the above analysis, we can recover the High(III) GS by GES, which is beneficial for recommendation. However, we identify an issue: High(III) and Low(I) with the same absolute value exhibit identical performance in recommendation, as shown in Table 2. This prompts us to consider how do graph signals of different frequencies affect recommendation. In fact, we find that low-frequency GS and high-frequency GS with the same absolute value have an equivalent impact on recommendation. Both influence the recommendation by affecting the similarity between user-item pairs, with the key lying in the waveform of the filter function.

Corollary 2 *The low-frequency graph signal Low(I) and the high-frequency graph signal High(III) with the same absolute value are equivalent in recommendation, and Low(IV) and High(II) are also equivalent.*

Proof of Corollary 2 As given in **Definition 3**, the filter function of Low(I), $f_{\text{I}}(\lambda) = \sum_{i=0}^{n}\alpha_i\lambda^i$, and the filter function of High(III), $f_{\text{III}}(\lambda) = \sum_{i=0}^{n}(-1)^{i+1}\alpha_i\lambda^i$. We perform graph embedding on it $\mathbf{E}_{\text{I}} = f_{\text{I}}(\lambda)\mathbf{E}^0$, $\mathbf{E}_{\text{III}} = f_{\text{III}}(\lambda)\mathbf{E}^0$. Then calculate the graph embedding signal. When calculating the graph embedding signal of High(III), we perform space flip on it. We can get the graph embedding signal of Low(I), $\mathbf{S}_{\text{I}} = \mathbf{E}_{\text{I}}\mathbf{E}_{\text{I}}^T$, and the graph embedding signal of High(III), $\mathbf{S}_{\text{III}} = -\mathbf{E}_{\text{III}}\mathbf{E}_{\text{III}}^T$. We expand $\mathbf{S}_{\text{I}}$ and $\mathbf{S}_{\text{III}}$ to get the following expression:

$$
\mathbf{S}_{\text{I}} = \begin{pmatrix} (1)^0\alpha_0\alpha_0\mathbf{E}^0(\mathbf{E}^0)^T + ... + (1)^n\alpha_0\alpha_n\mathbf{E}^0(\mathbf{E}^n)^T \\ +...+ \\ (1)^n\alpha_n\alpha_0\mathbf{E}^n(\mathbf{E}^0)^T + ... + (1)^{2n}\alpha_n\alpha_n\mathbf{E}^n(\mathbf{E}^n)^T \end{pmatrix} \tag{8}
$$

$$
\mathbf{S}_{\text{III}} = \begin{pmatrix} (-1)^3\alpha_0\alpha_0\mathbf{E}^0(\mathbf{E}^0)^T + ... + (-1)^{n+3}\alpha_0\alpha_n\mathbf{E}^0(\mathbf{E}^n)^T \\ +...+ \\ (-1)^{n+3}\alpha_n\alpha_0\mathbf{E}^n(\mathbf{E}^0)^T + ... + (-1)^{2n+3}\alpha_n\alpha_n\mathbf{E}^n(\mathbf{E}^n)^T \end{pmatrix} \tag{9}
$$

where $\mathbf{E}^n = \hat{\mathbf{A}}^n\mathbf{E}^0$. By comparison, we can find that the odd-order polynomial terms of $\mathbf{S}_{\text{I}}$ and $\mathbf{S}_{\text{III}}$ are the same, $\mathbf{S}_{ij}^{\text{odd(I)}} = \alpha_i\alpha_j\mathbf{E}^i\mathbf{E}^j$, $\mathbf{S}_{ij}^{\text{odd(III)}} = \alpha_i\alpha_j\mathbf{E}^i\mathbf{E}^j$, when the sum of the subscripts is odd. When the sum of the subscripts is even, their polynomials are opposite, $\mathbf{S}_{ij}^{\text{even(I)}} = \alpha_i\alpha_j\mathbf{E}^i\mathbf{E}^j$, $\mathbf{S}_{ij}^{\text{even(III)}} = -\alpha_i\alpha_j\mathbf{E}^i\mathbf{E}^j$. We use odd-order polynomials and even-order polynomials of Low(I) and High(III) with the same absolute value for recommendation respectively.

Table 3: The performance of the sum of odd-order and even-order polynomials of Low(I) and High(III) with the same absolute value on Recall@10.

Datasets	even(I)	odd(I)	even(III)	odd(III)
Gowalla	0.0754	0.1224	0.0754	0.1234
Yelp2018	0.0154	0.0451	0.0154	0.0447
Amazon-Books	0.0291	0.0438	0.0296	0.0448
Alibaba-iFasion	0.0330	0.0454	0.0334	0.0461

As shown in Table 3, we can find that the performance of odd-order odd(I) and odd(III) is very close, which is due to $\mathbf{S}_{ij}^{\text{odd(I)}} = \mathbf{S}_{ij}^{\text{odd(III)}}$. The even-order even(I) is opposite to even(III), $\mathbf{S}_{ij}^{\text{even(I)}} = -\mathbf{S}_{ij}^{\text{even(III)}}$, but the performance in recommendation is still very close. In fact, this is because the even-order polynomial $\mathbf{E}^i(\mathbf{E}^j)^T$ emphasizes the similarity between even-order node pairs on the graph, such as user-user and item-item pairs in recommendations, while the loss function in recommendation systems aims to minimize the similarity between user-item pairs. They are orthogonal and do not conflict, which leads to the equivalence of $\mathbf{S}_{ij}^{\text{even(I)}}$ and $\mathbf{S}_{ij}^{\text{even(III)}}$ in the training process of the recommendation model. Therefore, Low(I) and High(III) with the same absolute value are equivalent in recommendation, the same principle applies to Low(IV) and High(II). For details, please refer to the Appendix B.

4.2.1 Frequency Signal Scaler. From Table 3, we can see that the performance of **odd(I)** and **odd(III)** is much higher than that of **even(I)** and **even(III)**. This is due to the fact that the odd-order polynomial $\mathbf{E}^i(\mathbf{E}^j)^T$ has an important impact on the similarities between user-item pairs in recommendation, which directly affects the performance of recommendation. In fact, the waveform of the GNN's graph signal filter function $f(\lambda)$ will directly affect the weights of the odd-order polynomial coefficients. However, the current GNN-based recommendation model is still not flexible enough to adjust the waveform of $f(\lambda)$. Therefore, we propose a frequency signal scaler that can flexibly adjust the waveform of the original GNN filter function. As a plug-and-play plug-in, it can be integrated into other GNN models.

Inspired by the sigmoid function waveform, we propose a frequency signal scaling function $g(\lambda)$ to extract the frequency signal that needs to be adjusted and then scale it:

$$g(\lambda) = \frac{\mu}{1 + e^{\alpha(\lambda+\beta)}} \tag{10}$$

where $\alpha \in R$, $|\alpha|$ are used to control the steepness of the waveform. When $\alpha < 0$, it is used to extract low-frequency signals. When $\alpha > 0$, it is used to extract high-frequency signals. $\beta \in R$ is used to control the position of the waveform, μ is used to scale the size of the extracted waveform. Given a graph signal filter function $f(\lambda)$ of GNN, we multiply $g(\lambda)$ with it to obtain a scaled new graph signal filter function $f^{'}(\lambda) = g(\lambda) \cdot f(\lambda)$. Then we use the monomial bases to approximate the new graph signal filter function $f^{'}(\lambda)$:

$$f^{''}(\lambda) = \sum_{i=0}^{n} \alpha_i \lambda^i \tag{11}$$

where α_i is a learnable weight parameter and n represents the number of layers of the polynomial. We randomly sample some points $\mathbf{X}$ in the eigenvalue interval [-1,1] as training samples, and then input them into $f^{'}(\lambda)$ and $f^{''}(\lambda)$, and calculate the Euclidean distance between them as the loss function to train the coefficients α_i of the polynomial. To prevent interference between multi-objective optimization, we pre-train the coefficients of the graph signal filter function:

$$loss_{filter} = ||f^{'}(X) - f^{''}(X)||_2 \tag{12}$$

Using LightGCN as the base model, we apply the frequency signal scaler to it to adjust the waveform of its filter function. The experimental results are shown in Figure 3. The performance of LightGCN with different waveforms varies greatly, which is because the waveform of the filter function directly affects the similarity of user-item pairs.

5 Method

We propose a **Sim**ple spectral **G**raph **C**ollaborative **F**iltering model **SimGCF**. As shown in Figure 4, SimGCF can be divided into two versions: low-pass filtering and high-pass filtering. First, we select a GNN graph signal filter function as the backbone, and then we use the frequency signal scaler to adjust it more finely. In the high-pass filter version, we also need to perform space flip on it.

5.1 Backbone Graph Signal Filtering Function

The common graph signal filter function $f(\lambda)$ of the GNN-based recommendation model is a filter function in the form of a monomial

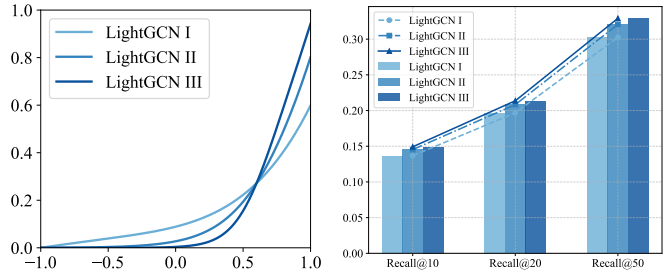


Figure 3: The performance of LightGCN with different waveforms on Gowalla.

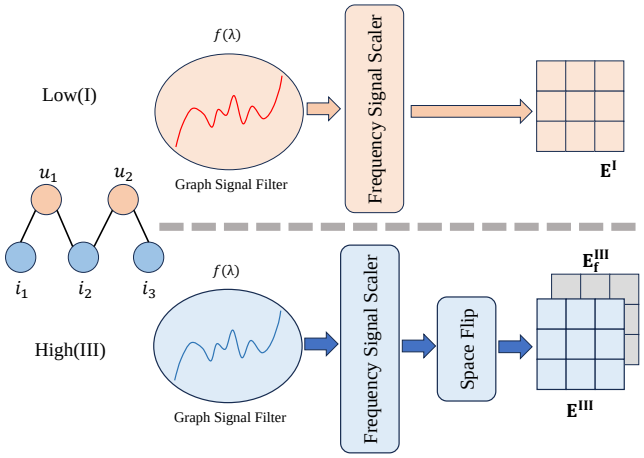


Figure 4: The framework of SimGCF. The low-frequency SimGCF performs frequency signal scaling on the original $f(\lambda)$, and the high-frequency SimGCF also needs to perform space flip on the embedding after scaling.

bases, such as LightGCN [9]. A study attempt to use a filter function based on Jacobi polynomial bases for graph collaborative filtering [6], which is in the following form:

$$f(\lambda) = \frac{1}{n+1} \sum_{i=0}^{n} \mathbf{J}_i^{a,b}(\lambda) \tag{13}$$

where $\mathbf{J}_i^{a,b}(\lambda)$ represents the i-th order Jacobi polynomial bases of λ. We choose Jacobi polynomial bases functions as our backbone graph signal filtering functions. In fact, our model is not limited to the Jacobi polynomial filter function, it can be replaced by any other form of filter function.

5.2 Frequency Signal Scaling

As described in 4.2.1, the waveform of $f(\lambda)$ directly affects the user-item similarity and has an important impact on the recommendation. However, the original polynomial filter function is still not flexible enough to adjust the graph signal, so we propose a frequency signal scaler $g(\lambda)$ to adjust it. The adjusted graph signal

filter function $f^{'}(\lambda)$ is as follows:

$$f^{'}(\lambda) = g(\lambda) \cdot f(\lambda) \tag{14}$$

We use a monomial bases, $f^{''}(\lambda) = \sum_{i=0}^{n} \alpha_i \lambda^i$, to approximate the adjusted filter function $f^{'}(\lambda)$ to learn the coefficients α_i. It can also be replaced by other polynomial bases. Then we express the original graph signal through graph embedding:

$$\mathbf{E} = \mathbf{U} f^{''}(\Lambda)\mathbf{U}^{\mathrm{T}}\mathbf{E}^0 = f^{''}(\hat{\mathbf{A}})\mathbf{E}^0 = \sum_{i=0}^{n} \alpha_i \hat{\mathbf{A}}^i \mathbf{E}^0 \tag{15}$$

Due to the low-frequency GES can directly express the GS of Low(I), there is no need to perform space flip, and $\mathbf{E}^{\mathbf{I}}$ can be obtained directly. However, High(III) GS cannot be directly expressed through GES, and the original high-frequency graph embedding $\mathbf{E}^{\mathbf{III}}$ needs to be spatially flipped $\mathbf{E}^{\mathbf{III}}_{\mathbf{f}} = -\mathbf{E}^{\mathbf{III}}$.

5.3 Optimization

We use the inner product of graph embedding to represent the interest score $\hat{y}_{ui}$ of user u and item i. On low-frequency SimGCF, $\hat{y}_{ui} = \mathbf{e}^{\mathbf{I}}_u * (\mathbf{e}^{\mathbf{I}}_i)^T$, where $\mathbf{e}^{\mathbf{I}}_u, \mathbf{e}^{\mathbf{I}}_i \in \mathbf{E}^{\mathbf{I}}$. On high-frequency SimGCF, $\hat{y}_{ui} = \mathbf{e}^{\mathbf{f}}_u * (\mathbf{e}^{\mathbf{III}}_i)^T$, where $\mathbf{e}^{\mathbf{f}}_u \in \mathbf{E}^{\mathbf{III}}_{\mathbf{f}}, \mathbf{e}^{\mathbf{III}}_i \in \mathbf{E}^{\mathbf{III}}$. Then we use Bayesian Personalized Rank (BPR) [19] as the recommendation loss:

$$\mathcal{L}_{bpr} = -\sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{N}_u} \sum_{j \notin \mathcal{N}_u} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) \tag{16}$$

where $\mathcal{N}_u$ is the set of neighbors of user u. The final loss function can be expressed as follows:

$$\mathcal{L} = \mathcal{L}_{bpr} + w\|\mathbf{E}^{\mathbf{0}}\|^2 \tag{17}$$

w controls the influence of the L_2-regularization term.

6 Experiments

We conduct experiments to answer the following questions: **Q1**: How does SimGCF compare to other baselines? **Q2**: What are the effects of space flip and frequency signals scaler in SimGCF? **Q3**: What is the impact of different hyper-parameters in SimGCF? **Q4**: What is the time complexity of SimGCF?

Table 5: Statistics of the Datasets.

Datasets	Users	Items	Interactions	Sparsity
Gowalla	29,858	40,981	1,027,464	99.92%
Amazon-Books	52,642	91,598	2,984,108	99.94%
Yelp2018	31,668	38,048	1,561,406	99.87%
Alibaba-iFashion	300,000	81,614	1,607,813	99.99%

6.1 Experimental Setup

6.1.1 Datasets. We conduct experiments on four widely used public datasets. Gowalla [4], Amazon-Books [16], Yelp [23] and Alibaba-iFashion [2]. The details of the datasets are shown in Table 5. We randomly sample 80% of the user interaction data for training, 10% of the data for validation, and the remaining 10% for testing.

6.1.2 Baselines. In our evaluation, SimGCF is benchmarked against a series of established baselines in the field. We follow the experimental setup outlined by Guo et al. [6], including hyperparameters and training protocols, to ensure consistency and comparability with established benchmarks. The baseline models include:

- **BPR** [19] is a matrix factorization framework based on Bayesian empirical loss.
- **NeuMF** [10] replaces the dot product in the MF model with a multi-layer perceptron to learn the match function of users and items.
- **NGCF** [23] adopts user-item bipartite graphs to incorporate high-order information and utilizes GNN to enhance CF.
- **DGCF** [24] produces disentangled representations for the user and item to improve the performance.
- **LightGCN** [9] simplifies the design of GCN to make it more concise and appropriate for recommendation.
- **GTN** [5] proposes to use graph trend filtering to denoise the user-item graph.
- **RGCF** [21] learns to denoise the user-item graph by removing noisy edges and then adding edges to ensure diversity.
- **DirectAU** [22] is a collaborative filtering method with representation alignment and uniformity.
- **JGCF** [6] is a spectral GNN based on Jacobi polynomial sets that can effectively utilize graph signals.

6.1.3 Evaluation Metrics. We use two widely used metrics, Recall@k and NDCG@k, to evaluate the performance of top-k recommendation. In our experiments, we set k to 10, 20 for reporting.

6.1.4 Implementation Details. Our experiments are based on the RecBole framework [27]. We optimize all baselines with Adam and carefully select hyperparameters according to their suggestions. The batch size is set to 4096 for Gowalla, Yelp, and Alibaba-iFashion, and the batch size is set to 8192 for Amazon-Books. All parameters are initialized by Xavier distribution. The embedding size of all methods is set to 64. Early stopping of 5 epochs is used to prevent overfitting, with Recall@20 as the metric. We use the parameter configuration of JGCF provided by the original paper. The experiments are based on a 32GB Tesla V100-PCIE GPU.

6.2 Overall Performance (Q1)

We use Low(I) and High(III) with the same absolute value as the filter function of SimGCF to obtain SimGCF(I) and SimGCF(III). The experimental results are shown in Table 4. From these results, we can draw the following conclusions:

Firstly, the GNN-based methods outperform traditional matrix factorization-based methods. The main reason is that the GNN-based model can utilize high-order information on the graph, compared to MF which can only use first-order interaction information for model training. Some recent work on denoising in GNN (GTN and RGCF) has achieved good results on most datasets. They further improve the performance of the model compared to LightGCN by eliminating the noise information in the interaction behavior. DirectAU achieves impressive performance by proposing a new loss for optimizing the uniformity and alignment of representations in recommendations. The above graph models are all based on GCN in the spatial domain and have poor processing capabilities for

Table 4: Main experimental results of SimGCF and baselines. The best result is bolded, and the runner-up is underlined.

Dataset	Metric	BPR	NeuMF	NGCF	DGCF	LightGCN	GTN	RGCF	DirectAU	JGCF*	**SimGCF(I)**	**SimGCF(III)**
Gowalla	Recall@10	0.1159	0.0975	0.1119	0.1252	0.1382	0.1403	0.1335	0.1394	0.1515	**0.1575**	0.1557
	NDCG@10	0.0811	0.0664	0.0787	0.0902	0.1003	0.1009	0.0905	0.0991	0.1106	**0.1136**	0.1119
	Recall@20	0.1686	0.1470	0.1633	0.1829	0.1983	0.2016	0.1934	0.2014	0.2172	**0.2222**	0.2183
	NDCG@20	0.0965	0.0808	0.0937	0.1066	0.1175	0.1184	0.1081	0.1170	0.1293	**0.1321**	0.1298
Amazon-Books	Recall@10	0.0477	0.0342	0.0475	0.0565	0.0620	0.0588	0.0712	0.0683	0.0746	**0.0803**	0.0792
	NDCG@10	0.0379	0.0266	0.0330	0.0448	0.0506	0.0485	0.0568	0.0569	0.0624	**0.0673**	0.0663
	Recall@20	0.0764	0.0575	0.076	0.0867	0.0953	0.0930	0.1090	0.1053	0.1144	**0.1202**	0.1196
	NDCG@20	0.0474	0.0345	0.0472	0.0551	0.0615	0.0597	0.0697	0.0689	0.0754	**0.0801**	0.0794
Yelp2018	Recall@10	0.0452	0.0313	0.0459	0.0527	0.0560	0.0603	0.0633	0.0557	0.0669	**0.0679**	0.0672
	NDCG@10	0.0355	0.0235	0.0364	0.0419	0.0450	0.0483	0.0503	0.0435	0.0541	**0.0550**	0.0549
	Recall@20	0.0764	0.0548	0.0778	0.0856	0.0913	0.0984	0.1026	0.0907	0.1066	0.1075	**0.1082**
	NDCG@20	0.046	0.0316	0.0472	0.0528	0.0569	0.0611	0.0637	0.0553	0.0673	0.0682	**0.0686**
Alibaba-iFashion	Recall@10	0.0303	0.0182	0.0382	0.0447	0.0477	0.0406	0.0450	0.0319	0.0559	**0.0571**	0.0570
	NDCG@10	0.0161	0.0092	0.0198	0.0241	0.0255	0.0217	0.0249	0.0166	0.0303	0.0310	**0.0311**
	Recall@20	0.0467	0.0302	0.0615	0.0677	0.0720	0.0625	0.0674	0.0484	0.0823	**0.0848**	0.0838
	NDCG@20	0.0203	0.0123	0.0257	0.0299	0.0316	0.0272	0.0305	0.0207	0.0370	**0.0380**	0.0379

Note: * means that only low-frequency and high-frequency graph signals are considered, and mid-frequency signals are removed.

graph signals. JGCF is a spectral GNN model based on the Jacobi polynomial bases. It controls the waveform of the graph signal filter by adjusting the values of hyperparameters a and b. It can more effectively learn signals of various frequencies on the graph and has demonstrated outstanding performance.

Moreover, SimGCF(I) consistently outperforms all baselines on all public datasets, which demonstrates the effectiveness of our proposed graph signal scaler. Due to the target of this paper is low-frequency and high-frequency graph signals, we choose to remove the mid-frequency signals in JGCF to eliminate the interference of mid-frequency signals. By comparing JGCF and SimGCF (I), we can find that it is beneficial to adjust the frequency signal of the original JGCF waveform. This is because the waveform of the graph signal filter function directly affects the similarity of user-item pairs in the recommendation. In fact, we find that the high-frequency signal in JGCF hinders the model's performance, which can be mitigated by filtering it out using a frequency signal scaler. This will be discussed in detail in the next section.

Finally, the performances of SimGCF(I) and SimGCF(III) are very close on all datasets. This demonstrates that the low-frequency graph signal Low (I) and the high-frequency graph signal High (III) with the same absolute value are equivalent in recommendation. Both influence the performance of the final recommendation by modulating the similarity between user-item pairs. Due to the limitations of current GES in capturing high-frequency GS beneficial for recommendation, we propose the space flip method to restores these signals through GES.

6.3 Ablation Studies (Q2)

To study the effects of space flip (SF) and frequency signal scaler (FSS) in SimGCF, we construct the following variants: JGCF, JGCF(H) and JGCF(L) represent the original JGCF, low-frequency JGCF and high-frequency JGCF respectively. JGCF(H)+SF means using SF for high-frequency JGCF. SimGCF(I) means performing FSS on the low-frequency JGCF, and SimGCF(III) means performing FSS and SF on the high-frequency JGCF.

Table 6: Ablation study of SimGCF. We abbreviate Recall and NDCG as R and N respectively.

Variants	**Gowalla**		**Amazon-Books**		**Yelp2018**		**Alibaba-iFasion**	
	R@20	N@20	R@20	N@20	R@20	N@20	R@20	N@20
JGCF	0.2172	0.1293	0.1144	0.0754	0.1066	0.0673	0.0823	0.0370
JGCF(H)	0.0383	0.0225	0.0098	0.0058	0.0494	0.0301	0.0222	0.0087
JGCF(H)+SF	0.1300	0.0754	0.0404	0.025	0.0522	0.0313	0.0639	0.0364
JGCF(L)	0.2212	0.1301	0.1167	0.0776	0.1068	0.0676	0.0834	0.0376
SimGCF(I)	0.2222	0.1321	0.1202	0.0801	0.1075	0.0682	0.0848	0.0380
SimGCF(III)	0.2183	0.1298	0.1196	0.0794	0.1082	0.0686	0.0838	0.0379

As shown in Table 6, by comparing JGCF, JGCF(H) and JGCF(L), we can find that the high-frequency signal in the original JGCF suppresses the expressiveness of the model. If we only retain the low-frequency signal, the performance of the model will improve. This is because GES cannot represent the high-frequency graph signals that are beneficial for recommendation. Consequently, the direct fusion of high-frequency and low-frequency signals hinders the model's performance.

Additionally, the comparison between JGCF(H)+SF and JGCF(H) indicates that SF effectively recovers the high-frequency graph signals that are beneficial for recommendation. Meanwhile, the performance comparison between JGCF(L) and SimGCF(I) shows that FSS remains advantageous for adjusting the filter function. Since the original low-frequency waveform of JGCF is already close to optimal, the performance improvement brought by FSS on JGCF is less significant than that on LightGCN. Lastly, the performance of SimGCF(I) is very similar to SimGCF(III), further confirming that high-frequency and low-frequency signals with the same absolute waveform are equivalent in recommendation systems.

6.4 Sensitivity Analysis (Q3)

We take SimGCF(I) as an example to perform parameter sensitivity analysis. The results of parameters μ, α, and β are shown in Figure 5. On the four datasets, we observe that the value of μ around 1 achieved the best performance. μ controls the scaling of the absolute value of $f(\lambda)$, and we observe that when $\mu < 1$, the performance of the model increases as the value of μ increases. When $\mu > 1$, the performance of the model decreases as the value of μ increases. This indicates that the waveform of the original graph signal filter function does not need to be scaled to a great extent.

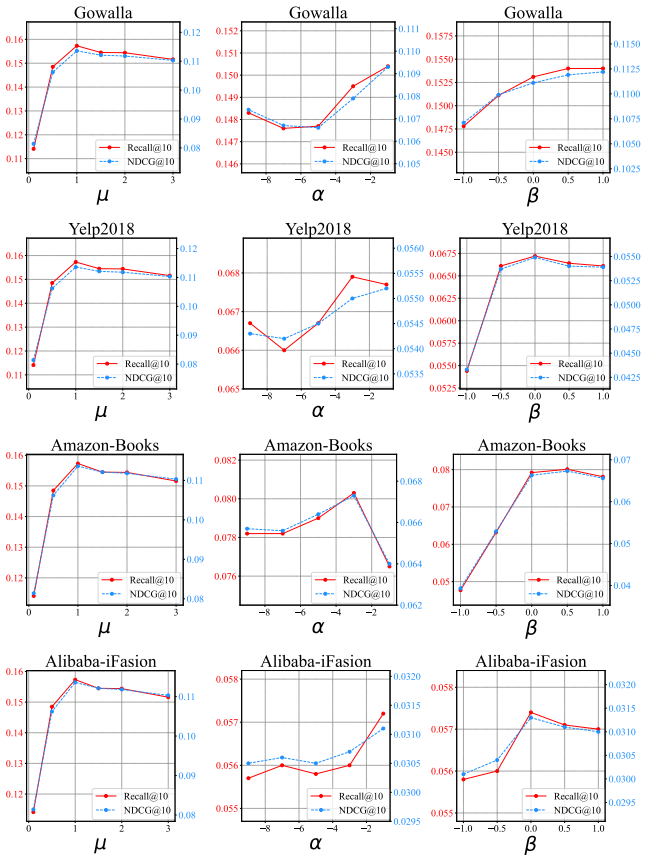


Figure 5: Analysis of μ, α and β.

The parameter α controls the steepness of the waveform of $f(\lambda)$, which has a greater impact on the performance of the model. We can observe that on Yelp2018 and Amazon-Books, the value of α reaches the optimal value around -3. When $\alpha < -3$, the performance of the model increases as the value of α increases. When $\alpha > -3$, the performance decreases as the value of α increases. On Alibaba-iFasion, $\alpha = -1$ reaches the optimal value. We observe that the steepness of the graph signal filter function has a significant impact on model performance. By adjusting the steepness of the original $f(\lambda)$, the similarity between user-item pairs is directly influenced.

The parameter β controls the adjustment range of the frequency signal scaler, which in turn significantly affects the model's overall performance. On the Gowalla, Yelp2018, Amazon-Books, and Alibaba-iFashion datasets, the best results are obtained when β is set to 0.5, 0, 0.5, and 0, respectively.

6.5 Complexity Analysis (Q4)

We explore the time complexity of SimGCF. SimGCF consists of two stages. The first stage involves the pre-training of the graph signal filtering function. The first stage uses a polynomial basis to approximate the filtering function and learn its coefficients, requiring no complex matrix operations and incurring negligible computational cost. In the second stage, we perform spectral graph convolution with the pre-trained coefficients. Since this stage involves matrix operations, we compare the per-epoch time complexity of SimGCF with other models. As shown in Figure 6, the overall efficiency of SimGCF is close to that of LightGCN, and is better than JGCF and NGCF. In fact, LightGCN can be considered a special case of SimGCF, as SimGCF employs the monomial bases.

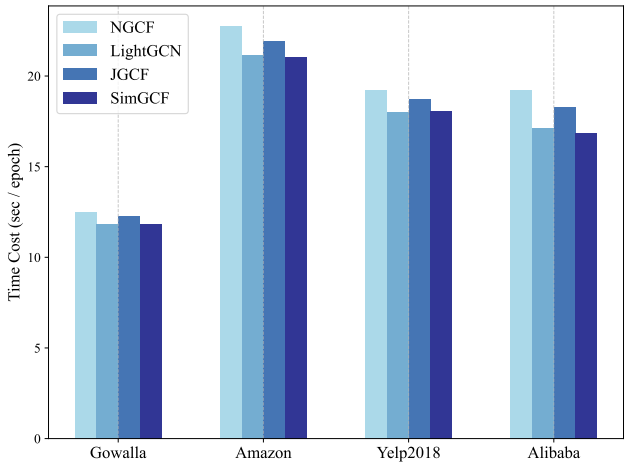


Figure 6: The time cost on each training epoch.

7 Conclusion

In this work, we investigate the influence of graph signals on recommender systems. We reveal and theoretically demonstrate that low-frequency and high-frequency graph signal filters with the same absolute value are equivalent in recommendation. They both affect recommendation by smoothing the similarities between user-item pairs. Furthermore, we identify that existing graph embedding methods can only express half of the characteristics of the graph signal. To address this, we propose SimGCF, a spectral GNN model with enhanced representation capabilities, offering greater flexibility in utilizing graph structure information. Extensive theoretical analysis and experiments validate the effectiveness of SimGCF.

Acknowledgments

This research was partially supported by the NSFC (62376180, 62506255, 62176175, 62176014), the Priority Academic Program Development of Jiangsu Higher Education Institutions and the Fundamental Research Funds for the Central Universities.

References

[1] Jingyu Chen, Runlin Lei, and Zhewei Wei. 2024. PolyGCL: Graph contrastive learning via learnable spectral polynomial filters. In *The Twelfth International Conference on Learning Representations*. OpenReview, online, 1–30f.

[2] Wen Chen, Pipei Huang, Jiaming Xu, Xin Guo, Cheng Guo, Fei Sun, Chao Li, Andreas Pfadler, Huan Zhao, and Binqiang Zhao. 2019. POG: personalized outfit generation for fashion recommendation at Alibaba iFashion. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. ACM, New York, NY, USA, 2662–2670.

[3] Zhiqian Chen, Fanglan Chen, Lei Zhang, Taoran Ji, Kaiqun Fu, Liang Zhao, Feng Chen, Lingfei Wu, Charu Aggarwal, and Chang-Tien Lu. 2023. Bridging the gap between spatial and spectral domains: A unified framework for graph neural networks. *Comput. Surveys* 56, 5 (2023), 1–42.

[4] Eunjoon Cho, Seth A Myers, and Jure Leskovec. 2011. Friendship and mobility: user movement in location-based social networks. In *Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, New York, NY, USA, 1082–1090.

[5] Wenqi Fan, Xiaorui Liu, Wei Jin, Xiangyu Zhao, Jiliang Tang, and Qing Li. 2022. Graph trend filtering networks for recommendation. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. ACM, New York, NY, USA, 112–121.

[6] Jiayan Guo, Lun Du, Xu Chen, Xiaojun Ma, Qiang Fu, Shi Han, Dongmei Zhang, and Yan Zhang. 2023. On Manipulating Signals of User-Item Graph: A Jacobi Polynomial-based Graph Collaborative Filtering. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, New York, NY, USA, 602–613.

[7] Yongjing Hao, Pengpeng Zhao, Jianfeng Qu, Lei Zhao, Guanfeng Liu, Fuzhen Zhuang, Victor S Sheng, and Xiaofang Zhou. 2024. Meta-optimized Structural and Semantic Contrastive Learning for Graph Collaborative Filtering. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, Utrecht, Netherlands, 679–691.

[8] Mingguo He, Zhewei Wei, and Ji-Rong Wen. 2022. Convolutional neural networks on graphs with chebyshev approximation, revisited. *Advances in neural information processing systems* 35 (2022), 7264–7276.

[9] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. ACM, New York, NY, USA, 639–648.

[10] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*. WWW, Republic and Canton of Geneva, CHE, 173–182.

[11] Wen-Zhi Li, Chang-Dong Wang, Hui Xiong, and Jian-Huang Lai. 2023. Homogcl: Rethinking homophily in graph contrastive learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, New York, NY, USA, 1341–1352.

[12] Cheng Liu, Chenhuan Yu, Ning Gui, Zhiwu Yu, and Songgaojun Deng. 2024. SimGCL: graph contrastive learning by finding homophily in heterophily. *Knowledge and Information Systems* 66, 3 (2024), 2089–2114.

[13] Fan Liu, Zhiyong Cheng, Lei Zhu, Zan Gao, and Liqiang Nie. 2021. Interest-aware message-passing GCN for recommendation. In *Proceedings of the web conference 2021*. ACM, New York, NY, USA, 1296–1305.

[14] Jiahao Liu, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, Li Shang, and Ning Gu. 2023. Personalized graph signal processing for collaborative filtering. In *Proceedings of the ACM Web Conference 2023*. ACM, New York, NY, USA, 1264–1272.

[15] Haitong Luo, Xuying Meng, Suhang Wang, Hanyun Cao, Weiyao Zhang, Yequan Wang, and Yujun Zhang. 2024. Spectral-Based Graph Neural Networks for Complementary Item Recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. AAAI, New York, NY, USA, 8868–8876.

[16] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. ACM, New York, NY, USA, 43–52.

[17] Shaowen Peng, Xin Liu, Kazunari Sugiyama, and Tsunenori Mine. 2024. How powerful is graph filtering for recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, New York, NY, USA, 2388–2399.

[18] Yifang Qin, Wei Ju, Xiao Luo, Yiyang Gu, Zhiping Xiao, and Ming Zhang. 2024. Polycf: Towards the optimal spectral graph filters for collaborative filtering. *arXiv preprint arXiv:2401.12590* abs/2401.12590 (2024), 1–10.

[19] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2012. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*. AUAI Press, Corvallis, Oregon, USA, 452–461. https://arxiv.org/abs/1205.2618

[20] Yifei Shen, Yongji Wu, Yao Zhang, Caihua Shan, Jun Zhang, B Khaled Letaief, and Dongsheng Li. 2021. How powerful is graph convolution for recommendation?. In *Proceedings of the 30th ACM international conference on information & knowledge management*. ACM, New York, NY, USA, 1619–1629.

[21] Changxin Tian, Yuexiang Xie, Yaliang Li, Nan Yang, and Wayne Xin Zhao. 2022. Learning to denoise unreliable interactions for graph collaborative filtering. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. ACM, New York, NY, USA, 122–132.

[22] Chenyang Wang, Yuanqing Yu, Weizhi Ma, Min Zhang, Chong Chen, Yiqun Liu, and Shaoping Ma. 2022. Towards representation alignment and uniformity in collaborative filtering. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. ACM, New York, NY, USA, 1816–1825.

[23] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*. ACM, New York, NY, USA, 165–174.

[24] Xiang Wang, Hongye Jin, An Zhang, Xiangnan He, Tong Xu, and Tat-Seng Chua. 2020. Disentangled graph collaborative filtering. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. ACM, New York, NY, USA, 1001–1010.

[25] Yili Wang, Kaixiong Zhou, Rui Miao, Ninghao Liu, and Xin Wang. 2022. Adagcl: Adaptive subgraph contrastive learning to generalize large-scale graph training. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. ACM, New York, NY, USA, 2046–2055.

[26] Jiafeng Xia, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, Li Shang, and Ning Gu. 2025. Frequency-aware graph signal processing for collaborative filtering. In *Companion Proceedings of the ACM on Web Conference 2025*. ACM, New York, NY, USA, 1417–1421.

[27] Lanling Xu, Zhen Tian, Gaowei Zhang, Junjie Zhang, Lei Wang, Bowen Zheng, Yifan Li, Jiakai Tang, Zeyu Zhang, Yupeng Hou, et al. 2023. Towards a more user-friendly and easy-to-use benchmark library for recommender systems. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 2837–2847.

[28] Yonghui Yang, Zhengwei Wu, Le Wu, Kun Zhang, Richang Hong, Zhiqiang Zhang, Jun Zhou, and Meng Wang. 2023. Generative-contrastive graph learning for recommendation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 1117–1126.

A GRAPH SIGNALS AND GRAPH EMBEDDING SIGNALS

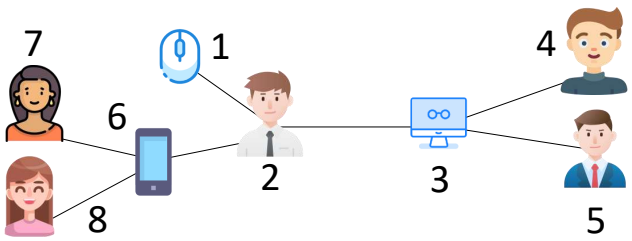


Figure 7: The case graph.

This section aims to explore the characteristics and differences between graph signals (GS) and graph embedding signals (GES). For visualization, we construct a small-scale case diagram **A**. Without loss of generality, our results can be extended to large-scale graphs. As shown in Figure 7, the graph contains 5 users and 3 products. We use four graph signal filter functions $f_{\mathrm{I}}(\lambda)$, $f_{\mathrm{II}}(\lambda)$, $f_{\mathrm{III}}(\lambda)$ and $f_{\mathrm{IV}}(\lambda)$ with the same absolute value in different quadrants to perform spectral operations on it, as shown in Figure 8. Among them, $f_{\mathrm{I}}(\lambda)$ and $f_{\mathrm{IV}}(\lambda)$ are low-pass filter functions, and $f_{\mathrm{II}(\lambda)}$ and $f_{\mathrm{III}}(\lambda)$ are high-pass filter functions.

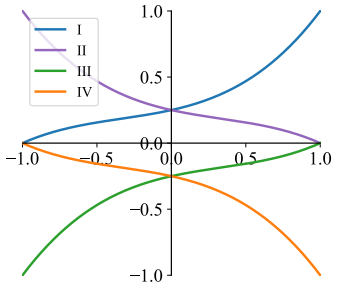


Figure 8: The waveforms of the graph signal filter function in four quadrants.

A.1 Graph Signals

The spectral operation of the graph signal is as follows:

$$\mathbf{S}_1 = f(\hat{\mathbf{A}}) = \sum_{i=0}^{n} \alpha_i \hat{\mathbf{A}} \tag{18}$$

where $\hat{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}$ represents the normalized adjacency matrix. n represents the order of the polynomial, which is set to 3 in our experiment. The characteristics of the GS obtained after filtering by the graph signal filter functions of four different quadrants are shown in Figure 9.

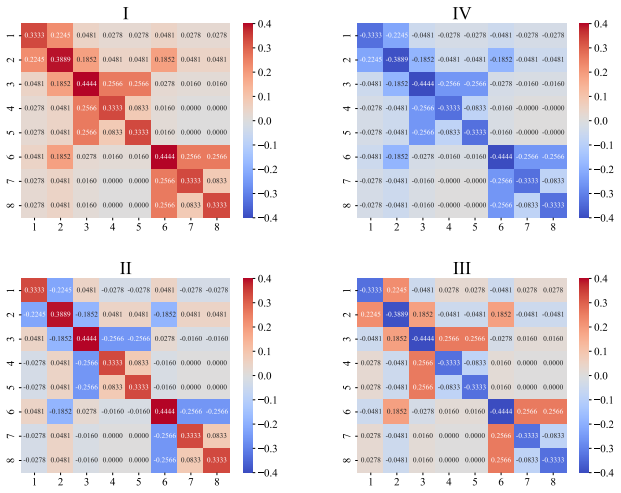


Figure 9: The characteristics of the graph signals in the four different quadrants.

From the experimental results, we can find that the low-frequency GS $f_{\mathrm{I}}(\lambda)$ and $f_{\mathrm{IV}}(\lambda)$ show the characteristics of stable changes, and the absolute value of the similarity between nodes decays with the increase of the path length. Taking the path 1->2->3->4 as an example, the similarities between node 1 and nodes 2, 3, and 4 are 0.2245, 0.0481, and 0.0278, respectively. When $f(\lambda) > 0$, the low-frequency GS $f_{\mathrm{I}}(\lambda)$ makes the connected nodes similar, while when $f(\lambda) < 0$, the low-frequency GS $f_{\mathrm{IV}}(\lambda)$ makes the connected nodes dissimilar. The high-frequency graph signals $f_{\mathrm{II}}(\lambda)$ and $f_{\mathrm{III}}(\lambda)$ show the characteristics of frequent changes. When $f(\lambda) > 0$, the high-frequency GS $f_{\mathrm{II}}(\lambda)$ will make odd-order nodes dissimilar and even-order nodes similar. As shown in the case graph, the similarities of odd-order nodes 1 and 2, 1 and 4 are -0.2245 and -0.0278 respectively, and the similarities of even-order nodes 2 and 4, 2 and 7 are 0.0481 and 0.0481 respectively. When $f(\lambda) < 0$, the high-frequency GS $f_{\mathrm{III}}(\lambda)$ will make odd-order nodes similar and even-order nodes dissimilar.

A.2 Graph Embedding Signals

The graph embedding signal is in the form of:

$$\mathbf{S}_2 = \mathbf{U}f(\Lambda)\mathbf{U}^{\mathrm{T}}\mathbf{E}^0 = f(\hat{\mathbf{A}})\mathbf{E}^0 = \sum_{i=0}^{n} \alpha_i \hat{\mathbf{A}}^i \mathbf{E}^0 \tag{19}$$

where $\mathbf{E}^0$ represents the initialized node embedding, and we use random initialization. n is still set to 3. The characteristics of the GES obtained after filtering by the graph signal filter functions in four different quadrants are shown in Figure 10.

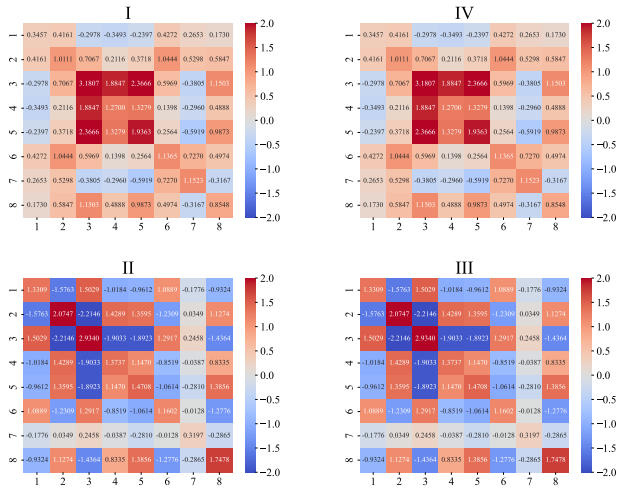


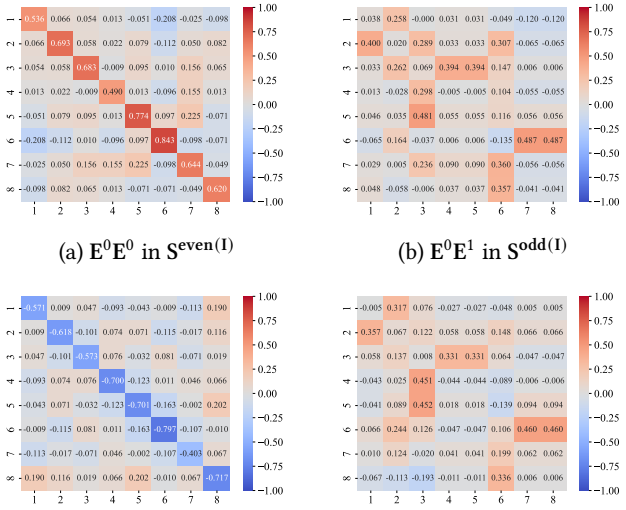
Figure 10: The characteristics of the graph embedding signals in the four different quadrants.

From the experimental results, we can find that the low-frequency GES $f_{\mathrm{I}}(\lambda)$ and $f_{\mathrm{IV}}(\lambda)$ show the characteristics of stable changes, and the absolute value of the similarity between nodes decays with the increase of the path length. The difference from the GS is that the features of $f_{\mathrm{I}}(\lambda)$ and $f_{\mathrm{IV}}(\lambda)$ of the GES are the same. This is because the GES cannot express the negative sign when calculating the similarity, resulting in the inability to express the feature of $f(\lambda) < 0$. Similarly, the high-frequency GES $f_{\mathrm{II}}(\lambda)$ and $f_{\mathrm{III}}(\lambda)$ cannot express the feature of $f(\lambda) < 0$.

B How DO GRAPH SIGNALS AFFECT RECOMMENDATION?

Let's take Low(I) and High(III) as examples. Given the filter functions $f_{\mathrm{I}}(\lambda)$ and $f_{\mathrm{III}}(\lambda)$ of Low(I) and High(III), as follows:

$$f_{\mathrm{I}}(\lambda) = \sum_{i=0}^{n} \alpha_i \lambda^i \tag{20}$$



(a) $\mathbf{E}^0\mathbf{E}^0$ in $\mathbf{S}^{\mathbf{even(I)}}$ (b) $\mathbf{E}^0\mathbf{E}^1$ in $\mathbf{S}^{\mathbf{odd(I)}}$

(c) $\mathbf{E}^0\mathbf{E}^0$ in $\mathbf{S}^{\mathbf{even(III)}}$ (d) $\mathbf{E}^0\mathbf{E}^1$ in $\mathbf{S}^{\mathbf{odd(III)}}$

Figure 11: The visualization results of $\mathbf{E}^0\mathbf{E}^0$ and $\mathbf{E}^0\mathbf{E}^1$ in graph embedding signals $\mathbf{S}_\mathbf{I}$ and $\mathbf{S}_\mathbf{III}$.

$$f_{\mathrm{III}}(\lambda) = \sum_{i=0}^{n} (-1)^{i+1} \alpha_i \lambda^i \tag{21}$$

where n represents the number of layers of the polynomial bases. We embed them, $\mathbf{E}_\mathbf{I} = f_\mathrm{I}(\lambda)\mathbf{E}^0$, $\mathbf{E}_\mathbf{III} = f_\mathrm{III}(\lambda)\mathbf{E}^0$. Then we calculate the graph embedding signals $\mathbf{S}_\mathbf{I}$ and $\mathbf{S}_\mathbf{III}$ of Low(I) and High(III) as follows. Note that we need to perform space flip when calculating $\mathbf{S}_\mathbf{III}$:

$$\begin{aligned} \mathbf{S}_\mathbf{I} &= \mathbf{E}_\mathbf{I}\mathbf{E}_\mathbf{I}^T \\ &= \begin{pmatrix} (1)^0\alpha_0\alpha_0\mathbf{E}^0(\mathbf{E}^0)^T + \ldots + (1)^n\alpha_0\alpha_n\mathbf{E}^0(\mathbf{E}^n)^T \\ + \ldots + \\ (1)^n\alpha_n\alpha_0\mathbf{E}^n(\mathbf{E}^0)^T + \ldots + (1)^{2n}\alpha_n\alpha_n\mathbf{E}^n(\mathbf{E}^n)^T \end{pmatrix} \end{aligned} \tag{22}$$

$$\begin{aligned} \mathbf{S}_\mathbf{III} &= -\mathbf{E}_\mathbf{III}\mathbf{E}_\mathbf{III}^T \\ &= \begin{pmatrix} (-1)^3\alpha_0\alpha_0\mathbf{E}^0(\mathbf{E}^0)^T + \ldots + (-1)^{n+3}\alpha_0\alpha_n\mathbf{E}^0(\mathbf{E}^n)^T \\ + \ldots + \\ (-1)^{n+3}\alpha_n\alpha_0\mathbf{E}^n(\mathbf{E}^0)^T + \ldots + (-1)^{2n+3}\alpha_n\alpha_n\mathbf{E}^n(\mathbf{E}^n)^T \end{pmatrix} \end{aligned} \tag{23}$$

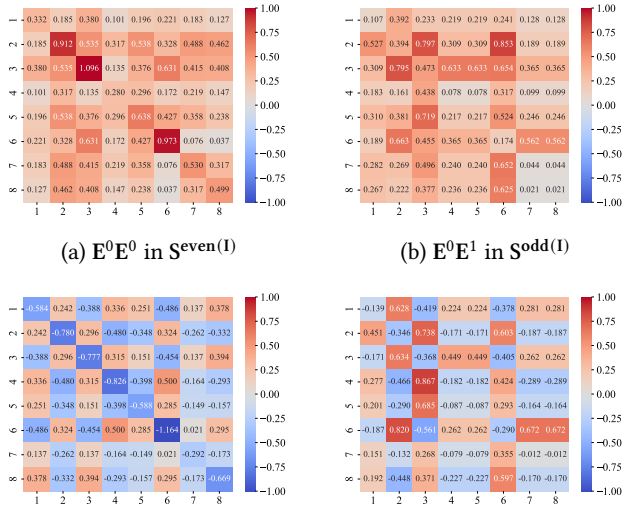
where $\mathbf{E}^n = \hat{\mathbf{A}}^n\mathbf{E}^0$ represents the node embedding of the n-th layer. We can extract the polynomials with odd-order subscript sums $\mathbf{S}_{ij}^{\mathbf{odd(I)}} = \alpha_i\alpha_j\mathbf{E}^i\mathbf{E}^j$, $\mathbf{S}_{ij}^{\mathbf{odd(III)}} = \alpha_i\alpha_j\mathbf{E}^i\mathbf{E}^j$ from $\mathbf{S}_\mathbf{I}$ and $\mathbf{S}_\mathbf{III}$. It is not difficult to see that them are equal $\mathbf{S}_{ij}^{\mathbf{odd(I)}} = \mathbf{S}_{ij}^{\mathbf{odd(III)}}$. We can extract the polynomials with even-order subscript sums $\mathbf{S}_{ij}^{\mathbf{even(I)}} = \alpha_i\alpha_j\mathbf{E}^i\mathbf{E}^j$, $\mathbf{S}_{ij}^{\mathbf{even(III)}} = -\alpha_i\alpha_j\mathbf{E}^i\mathbf{E}^j$ from $\mathbf{S}_\mathbf{I}$ and $\mathbf{S}_\mathbf{III}$, where $\mathbf{S}_{ij}^{\mathbf{even(I)}} = -\mathbf{S}_{ij}^{\mathbf{even(III)}}$. To prove that $\mathbf{S}_\mathbf{I} = \mathbf{S}_\mathbf{III}$ in the recommendation, we only need to prove that $\mathbf{S}^{\mathbf{even(I)}} = \mathbf{S}^{\mathbf{even(III)}}$ in the recommendation.

In fact, we find that $\mathbf{S}^{\mathbf{even(I)}}$ will increase the similarity between even-order neighbors on the graph, $\mathbf{S}^{\mathbf{even(III)}}$ will reduce the similarity between even-order neighbors on the graph. We take $\mathbf{E}^0(\mathbf{E}^0)^T$ and $\mathbf{E}^0(\mathbf{E}^1)^T$ as examples, as shown in Figure 11. We find that $\mathbf{E}^0(\mathbf{E}^0)^T$ in $\mathbf{S}^{\mathbf{even(I)}}$ emphasizes the similarity between the 0th-order node and its 0-order neighbors. For example, the similarity between node 1 and itself is 0.536. $\mathbf{E}^0(\mathbf{E}^0)^T$ in $\mathbf{S}^{\mathbf{even(III)}}$ emphasizes the dissimilarity between the 0-order node and its 0-order neighbors. For example, the similarity between node 1 and itself is -0.576.

The loss function in recommendation emphasizes the similarity between users and items they have been exposed to, such as the BPR [19] loss:

$$\mathcal{L}_{bpr} = -\sum_{u\in\mathcal{U}}\sum_{i\in\mathcal{N}_u}\sum_{j\notin\mathcal{N}_u} \ln\sigma(\hat{y}_{ui} - \hat{y}_{uj}) \tag{24}$$

where $\hat{y}_{ui}$ represents the similarity between user u and item i. $\mathcal{U}$ represents the set of users, and $\mathcal{N}_u$ represents the set of items that the user interacts with. BPR loss increases the similarity between interactive user-item pairs and suppresses the similarity between non-interactive user-item pairs, which is consistent with the purpose of recommendation. The $\mathbf{S}^{\mathbf{even(I)}}$ and $\mathbf{S}^{\mathbf{even(III)}}$ both emphasize the relationship between even-order neighbors, which does not conflict with the goal of recommendation. As shown in Figure 12, we use BPR loss to train $\mathbf{E}^0\mathbf{E}^0$ in $\mathbf{S}^{\mathbf{even(I)}}$ and $\mathbf{S}^{\mathbf{even(III)}}$. We can find that the original graph structure information and the information of BPR can be well preserved on $\mathbf{E}^0\mathbf{E}^0$, and they are orthogonal. During the training process of the recommendation model, them are equivalent, i.e. $\mathbf{S}^{\mathbf{even(I)}} = \mathbf{S}^{\mathbf{even(III)}}$ in recommendation. Therefore, the low-frequency grapn signal $\mathbf{S}_\mathbf{I}$ and the high-frequency grapn signal $\mathbf{S}_\mathbf{III}$ with the same absolute value have equal effects on the recommendation.



(a) $\mathbf{E}^0\mathbf{E}^0$ in $\mathbf{S}^{\mathbf{even(I)}}$ (b) $\mathbf{E}^0\mathbf{E}^1$ in $\mathbf{S}^{\mathbf{odd(I)}}$

(c) $\mathbf{E}^0\mathbf{E}^0$ in $\mathbf{S}^{\mathbf{even(III)}}$ (d) $\mathbf{E}^0\mathbf{E}^1$ in $\mathbf{S}^{\mathbf{odd(III)}}$

Figure 12: The visualization results of training $\mathbf{E}^0\mathbf{E}^0$ and $\mathbf{E}^0\mathbf{E}^1$ in graph embedding signals $\mathbf{S}_\mathbf{I}$ and $\mathbf{S}_\mathbf{III}$ using BPR.